

BÁO CÁO BÀI TẬP LỚN PYTHON

Đề tài: Hệ thống Thu thập, Tra cứu và Phân tích Dữ liệu Cầu thủ Ngoại hạng Anh 2024-2025

I. TỔNG QUAN DỰ ÁN

1. Mục tiêu:

- Xây dựng hệ thống end-to-end từ cào dữ liệu (Web Scraping), lưu trữ cơ sở dữ liệu (SQLite), xây dựng API (Flask) đến phân tích dữ liệu (Pandas) và học máy (K-Means, PCA).

2. Công cụ & Thư viện sử dụng:

- Thu thập dữ liệu: `requests`, `BeautifulSoup4`, `undetected_chromedriver`, `selenium`.
- Backend & Database: `Flask`, `sqlite3`.
- Phân tích & Trực quan hóa: `pandas`, `numpy`, `matplotlib`, `seaborn`, `rich`.
- Học máy: `scikit-learn` (KMeans, PCA, StandardScaler, silhouette_score).

II. QUÁ TRÌNH THỰC HIỆN & PHƯƠNG PHÁP GIẢI QUYẾT

1. Thu thập và lưu trữ dữ liệu (Yêu cầu I)

Tác dụng file scraper.py: Đóng vai trò là module Data Engineering, chịu trách nhiệm tự động hóa quy trình trích xuất dữ liệu (ETL: Extract - Transform - Load). File xử lý việc vượt tường lửa (anti-bot), bóc tách dữ liệu phức tạp từ DOM/Javascript và tổ chức lưu trữ an toàn vào cơ sở dữ liệu.

1.1. Scraping fbref.com (Câu I.1):

- **Thách thức:** Fbref sử dụng hệ thống chống bot Cloudflare cực kỳ nghiêm ngặt và ẩn bảng dữ liệu trong các đoạn chú thích (HTML Comment) để tăng tốc độ tải trang.
- **Giải thích code & thuật toán:**
 - **Vượt Cloudflare:** Khởi tạo trình duyệt ẩn danh bằng thư viện `undetected_chromedriver`. Thuật toán sử dụng vòng lặp `while True` kết hợp `time.sleep` để quét liên tục mã nguồn HTML, chờ đến khi dòng chữ "*Just a moment*" biến mất, đảm bảo đã vượt qua bài kiểm tra của Cloudflare trước khi lấy `page_source`.
 - **Bóc tách dữ liệu ẩn:** Sử dụng `BeautifulSoup` kết hợp hàm `lambda` để duyệt qua toàn bộ các HTML Comment, tìm và giải nén bảng có `id="stats_standard"`.
 - **Làm sạch dữ liệu (Data Cleaning):** Cột quốc tịch (`nation`) chứa nhiều ký tự thừa, nhóm sử dụng thư viện `re` (Regex) để trích xuất chính xác 3 chữ cái mã quốc gia (VD: ENG, FRA). Sau đó, dùng khối lệnh `try...except` và điều kiện `if` lọc bỏ toàn bộ các cầu thủ thi đấu ≤ 90 phút theo đúng yêu cầu đề bài.

1.2. Scraping footballtransfers.com (Câu I.2):

- **Thách thức:** Trang web có giới hạn rate-limit chặt chẽ, dễ bị khóa IP nếu search liên tục. Dữ liệu định giá cầu thủ không nằm ở các thẻ HTML thông thường mà bị mã hóa.
- **Giải thích code & thuật toán:**
 - **Thuật toán Ánh xạ (Mapping):** Thay vì search từng cầu thủ dễ bị chặn, hàm `build_player_map()` đi một đường vòng thông minh: truy cập danh sách các đội Ngoại hạng Anh, duyệt qua trang chuyển nhượng của từng đội và trích xuất URL gốc của toàn bộ cầu thủ. Để tránh lỗi sai chính tả khi ghép nối dữ liệu giữa 2 trang web, nhóm dùng hàm `normalize()` kết hợp thư viện `unicodedata` loại bỏ dấu tiếng Việt, ký tự đặc biệt và sắp xếp lại chuỗi tên.
 - **Dịch ngược dữ liệu (Reverse-engineering):** Để lấy định giá (Estimated Transfer Value) mà không cần dùng Selenium mô

phỏng thao tác render biểu đồ rất chậm, nhóm áp dụng 2 thuật toán bóc tách trực tiếp từ mã nguồn:

- Tìm thẻ chứa thuộc tính `data-base64`, dùng thư viện `base64` và `json` giải mã chuỗi này thành dạng từ điển (Dictionary) để lấy biểu đồ giá.
- Sử dụng Regex (`re.search`) tìm biến `chartData` nằm sâu trong các thẻ `<script>`, tải cấu trúc JSON tĩnh và dùng vòng lặp trích xuất giá trị lớn nhất trong năm 2024-2025.

1.3. Lưu trữ SQLite:

- **Giải thích code & thuật toán:**
 - **Bảng `player_stats`:** Chuyển đổi dữ liệu đã cào thành `pd.DataFrame`, lấp đầy các giá trị rỗng bằng `fillna("N/a")`, sau đó dùng hàm `to_sql()` để đẩy nguyên khối dữ liệu vào DB. Nhóm thực thi thêm lệnh SQL `CREATE INDEX idx_player` để tối ưu hóa thời gian chạy các truy vấn `JOIN` sau này.
 - **Bảng `player_values`:** Quá trình cào giá có thể mất nhiều thời gian và dễ bị gián đoạn do mạng. Nhóm thiết lập khóa chính (Primary Key) cho cột `player` và dùng câu lệnh `INSERT OR REPLACE INTO`. Thuật toán Upsert này giúp tiến trình có thể tạm dừng và chạy tiếp mà không bao giờ sinh ra dữ liệu rác hay bị trùng lặp dòng.

2. Xây dựng Restful API và Công cụ tra cứu (Yêu cầu II)

Cấu trúc tệp tin: Chức năng này được chia làm 2 file hoạt động độc lập theo mô hình Client - Server: `api_app_2.py` đóng vai trò Server cung cấp dữ liệu, và `lookup_2.py` đóng vai trò Client (CLI) gọi dữ liệu để hiển thị.

2.1. Server API (`api_app_2.py`):

- **Tác dụng:** Khởi tạo một web server cục bộ bằng mô-đun Flask để lắng nghe các yêu cầu truy xuất dữ liệu.
- **Giải thích code & thuật toán:**
 - Khởi tạo endpoint `/api/players` tiếp nhận phương thức `GET`.
 - Thuật toán tìm kiếm sử dụng truy vấn SQL động. Thay vì code cứng điều kiện, nhóm sử dụng kỹ thuật `WHERE 1=1` kết hợp

với toán tử **LIKE** để linh hoạt nối thêm các chuỗi tham số (Tên cầu thủ **name** hoặc Câu lạc bộ **club**) nếu người dùng có truyền vào.

- Sử dụng **LEFT JOIN** trong SQL để kết nối bảng **player_stats** và **player_values** thông qua khóa **player**, đảm bảo lấy được giá chuyển nhượng (nếu có).
- Sử dụng **sqlite3.Row** để chuyển đổi kết quả truy vấn thành dạng từ điển (dictionary) và dùng **jsonify()** để trả về client dưới định dạng chuẩn JSON.

2.2. Công cụ tra cứu CLI (**lookup_2.py**):

- **Tác dụng:** Cung cấp giao diện dòng lệnh cho người dùng tra cứu thông tin mà không cần thao tác trực tiếp với cơ sở dữ liệu.
- **Giải thích code & thuật toán:**
 - Sử dụng thư viện **argparse** để định nghĩa và bắt các tham số đầu vào (**--name**, **--club**) từ terminal.
 - Sử dụng mô-đun **requests** để gửi HTTP GET request kèm params lên Server API.
 - Khi nhận được JSON trả về, nhóm sử dụng thư viện **pandas** (**pd.DataFrame**) để nạp dữ liệu. Dữ liệu sau đó được định tuyến qua một mảng **preferred_order** để sắp xếp lại các cột thông tin sao cho các chỉ số quan trọng (tuổi, vị trí, bàn thắng, giá trị) hiện lên trước.
 - Để tối ưu UI/UX trên terminal, nhóm sử dụng thư viện **rich.table** vòng lặp qua từng hàng (row) của DataFrame để vẽ bảng dữ liệu hiển thị trực quan, thay vì chỉ in raw text. Đồng thời, hàm **to_csv()** được gọi để kết xuất dữ liệu ra file lưu vào thư mục **outputs/**.

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
----------	--------	---------------	----------	-------

KẾT QUẢ TRA CỨU: Arsenal

Player	Position	Age	Games	Goals	Assists	Transfer value
Calafiori Riccardo	DF	22	19	2	1	53883002
Havertz Kai	FW	25	23	9	3	69957843
Jesus Gabriel	FW	27	17	3	0	59130000
Jorginho	MF	32	15	0	0	N/a
Kiwior Jakub	DF	24	17	1	0	N/a
Lewis-Skelly Myles	DF	17	23	1	0	60582810
Magalhães Gabriel	DF	26	28	3	1	76230583
Martinelli Gabriel	FW,MF	23	33	8	4	82739476
Merino Mikel	MF,FW	28	28	7	2	51642471
Nwaneri Ethan	FW,MF	17	26	4	2	N/a
Partey Thomas	MF,DF	31	35	4	2	N/a
Raya David	GK	28	38	0	0	47770298
Rice Declan	MF	25	35	4	7	121119286
Saka Bukayo	FW,MF	22	25	6	10	145949345
Saliba William	DF	23	35	2	0	79520000
Sterling Raheem	FW,MF	29	17	0	2	N/a

3. Phân tích Thống kê và Định giá (Yêu cầu III.1 & III.2)

Tác dụng file `analysis_4.py`: Đóng vai trò là module Data Science cốt lõi, chịu trách nhiệm làm sạch dữ liệu (Data Cleaning), phân tích thống kê (Descriptive Statistics), định giá bằng thuật toán tự xây dựng và áp dụng mô hình Học máy.

3.1. Thuật toán Thống kê (Câu III.1):

- **Làm sạch dữ liệu:** Sử dụng `pd.read_sql` nạp dữ liệu. Các giá trị rỗng hoặc N/a được xử lý bằng `df.replace` và `fillna(0)`. Chuyển đổi toàn bộ cột số liệu sang định dạng numeric bằng `pd.to_numeric` để phục vụ tính toán.

- **Thống kê mô tả:** Sử dụng hàm `groupby('team')` kết hợp với `agg(['mean', 'median', 'std'])` để tính trung bình, trung vị và độ lệch chuẩn cho từng chỉ số của từng đội trong một lệnh duy nhất, giúp tối ưu hiệu năng.
- **Đánh giá phong độ:** Thuật toán duyệt qua tập hợp các chỉ số trọng yếu (bàn thắng, kiến tạo, thẻ phạt). Dùng hàm `sum()` để tính tổng toàn đội, sau đó tìm index cao nhất (`idxmax()`) cho các chỉ số tích cực (ghi bàn) và index thấp nhất (`idxmin()`) cho chỉ số tiêu cực (thẻ phạt). Nhóm sử dụng một cấu trúc dữ liệu Dictionary (`best_teams_count`) để đếm tần suất dẫn đầu của các đội, từ đó tự động xuất ra kết luận đội có phong độ tốt nhất.

```

--- Đang xử lý III.1: Thống kê đội bóng ---
✅ Đã xuất file Team_Statistics.csv

[Đội bóng dẫn đầu các chỉ số quan trọng]
- Nhiều goals nhất: Liverpool (85)
- Nhiều assists nhất: Liverpool (65)
- Nhiều minutes nhất: Nottingham Forest (37569)
- Ít cards_yellow nhất: Manchester City (59 thẻ)
- Ít cards_red nhất: Leicester City (0 thẻ)

=> KẾT LUẬN (Câu III.1): Đội có phong độ tốt nhất giải là **Liverpool**.

```

3.2. Thuật toán Định giá (Câu III.2):

- **Giải thích code:** Khởi tạo hàm `calculate_value` và áp dụng (apply) lên từng hàng (axis=1) của DataFrame.
- **Luồng xử lý:** Thuật toán sử dụng cấu trúc rẽ nhánh (`if/elif/else`) phân rã logic tính điểm theo trường `position` (Vị trí). Trọng số tính điểm được gán khác nhau (VD: Tiền đạo/Tiền vệ tập trung nhân hệ số vào số bàn thắng/kiến tạo, trong khi Hậu vệ/Thủ môn tính hệ số dựa trên số trận đá chính `games_starts`).
- Hàm `max(0.5, value)` được sử dụng ở bước cuối để đặt mốc chặn dưới, đảm bảo không có lỗi logic định giá cầu thủ rơi vào mức âm.

Đề xuất phương pháp định giá cầu thủ

Dựa trên tệp dữ liệu đã thu thập từ câu I, nhóm đề xuất một mô hình định giá cầu thủ tuyến tính dựa trên 3 trụ cột chính: Sự đóng góp, Tiềm năng (Độ tuổi) và Đặc thù vị trí thi đấu.

Công thức đề xuất: Giá trị ước tính (Triệu) = Điểm cơ bản + Điểm tiềm năng + Điểm thành tích

Trong đó:

1. **Điểm cơ bản (Thời lượng thi đấu):** Tính bằng $(\text{Số phút thi đấu} / 90) * 0.5$. Cầu thủ ra sân càng nhiều chứng tỏ vai trò không thể thay thế trong đội hình, giá trị cơ bản càng cao.
2. **Điểm tiềm năng (Độ tuổi):** Tính bằng $\text{Max}(0, (28 - \text{Tuổi}) * 0.5)$. Trong bóng đá, độ tuổi 28 thường là đỉnh cao phong độ. Những cầu thủ dưới 28 tuổi sẽ được cộng thêm giá trị tiềm năng (phí chuyển nhượng cao hơn do thời gian cống hiến còn dài). Cầu thủ trên 28 tuổi không được cộng điểm này.
3. **Điểm thành tích (Dựa theo Vị trí):**
 - Đối với Tiền đạo/Tiền vệ (FW, MF): Đánh giá qua mũi nhọn tấn công: $(\text{Bàn thắng} * 2.0) + (\text{Kiến tạo} * 1.5)$.
 - Đối với Hậu vệ/Thủ môn (DF, GK): Vì không có dữ liệu cản phá trong tệp Câu I, nhóm đánh giá qua độ ổn định nơi hàng thủ: $\text{Số trận đá chính (games_starts)} * 1.0$.

Lưu ý: Tổng giá trị được giới hạn mức tối thiểu là 0.5 để đảm bảo không có cầu thủ nào bị định giá âm hoặc bằng 0.

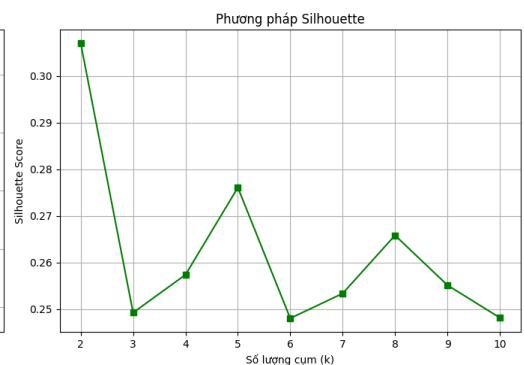
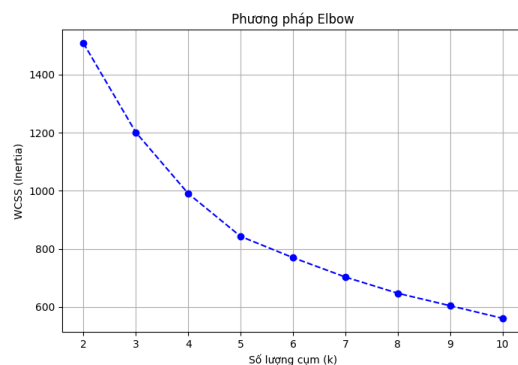
[KẾT QUẢ III.2] TOP 10 CẦU THỦ ĐƯỢC ĐỊNH GIÁ CAO NHẤT THEO MÔ HÌNH:

player	team	position	age	Estimated_Value_M
Salah Mohamed	Liverpool	MF	32	103.73
Isak Alexander	Newcastle United	FW	24	72.31
Mbeumo Bryan	Brentford	MF,FW	24	71.47
Haaland Erling	Manchester City	FW	24	65.70
Palmer Cole	Chelsea	MF	22	62.73
Wood Chris	Nottingham Forest	FW	32	60.94
Wissa Yoane	Brentford	FW	27	60.72
Kerkez Milos	Bournemouth	DF	20	60.53
Collins Nathan	Brentford	DF	23	59.50
Watkins Ollie	Aston Villa	FW	28	58.43

4. Phân cụm Học máy K-Means & PCA (Yêu cầu III cuối)

4.1. Giải thích Thuật toán Phân cụm (K-Means):

- Nhóm trích xuất 4 chiều đặc trưng (features) quan trọng: **goals_per90**, **assists_per90**, **minutes_90s**, và **age**.
- **Chuẩn hóa dữ liệu:** Sử dụng **StandardScaler** để đưa các giá trị về phân phối chuẩn (mean = 0, std = 1). Đây là bước bắt buộc vì thuật toán K-Means sử dụng khoảng cách Euclidean; nếu không chuẩn hóa, các biến có thang đo lớn (như số phút thi đấu) sẽ lấn át các biến có thang đo nhỏ (như số bàn/90p).
- Sử dụng vòng lặp để test mô hình với số cụm k chạy từ 2 đến 10. Thuật toán K-Means của **scikit-learn** sẽ khởi tạo ngẫu nhiên trọng tâm, gán nhãn dữ liệu và cập nhật trọng tâm liên tục cho đến khi hội tụ. Tại mỗi bước, nhóm tính toán độ lỗi WCSS (Inertia) và hệ số Silhouette để vẽ đồ thị tìm k tối ưu.



Dựa vào kết quả chạy thuật toán hiển thị trên 2 biểu đồ, nhóm quyết định phân loại cầu thủ thành **4 nhóm (k=4)**.

- **Lý do (Dựa trên biểu đồ Elbow):** Quan sát đồ thị WCSS (phương sai nội cụm), ta thấy đường gấp khúc bắt đầu giảm độ dốc và thoải dần ra tại vị trí $k=3$ và $k=4$.
- **Lý do (Dựa trên biểu đồ Silhouette):** Đối chiếu sang hệ số Silhouette, tại $k=4$, điểm số vẫn giữ được ở mức tương đối khả quan so với các giá trị k lớn hơn, cho thấy các điểm dữ liệu trong cùng một cụm có độ gắn kết tốt và tách biệt tương đối rõ ràng với các cụm khác. Việc chọn $k=4$ giúp mô hình không bị quá chung chung (k quá nhỏ) cũng không bị phân mảnh vụn vặt (k quá lớn).

4.2. Nhận xét kết quả phân cụm:

- Nhóm 0 (Nhóm kép phụ/Cầu thủ trẻ): Thường có số phút thi đấu thấp, đóng góp vào bàn thắng/kiến tạo chưa nhiều.
- Nhóm 1 (Nhóm Tiền vệ tấn công/Chạy cánh): Có chỉ số kiến tạo trên 90 phút (assists_per90) nổi trội.
- Nhóm 2 (Nhóm Tiền đạo mục tiêu): Nổi bật hoàn toàn với chỉ số ghi bàn (goals_per90) cao nhất trong 4 nhóm.
- Nhóm 3 (Nhóm Hậu vệ/Tiền vệ đánh chặn): Cầu thủ có độ tuổi chín chắn, thi đấu gần như trọn vẹn số phút nhưng các chỉ số tấn công (bàn thắng, kiến tạo) rất thấp do đặc thù vị trí.

[KẾT QUẢ III.3] ĐẶC ĐIỂM TRUNG BÌNH CỦA 4 NHÓM CẦU THỦ (CLUSTERS):

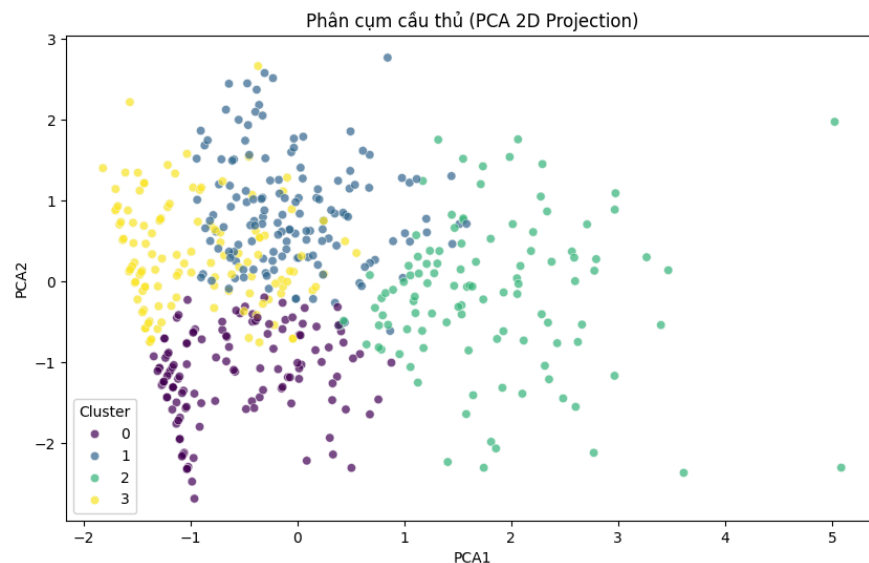
	age	minutes	goals	assists
Cluster				
0	21.47	766.76	0.52	0.45
1	26.05	2613.45	1.74	1.94
2	24.94	1603.74	6.49	3.89
3	29.70	815.32	0.50	0.37

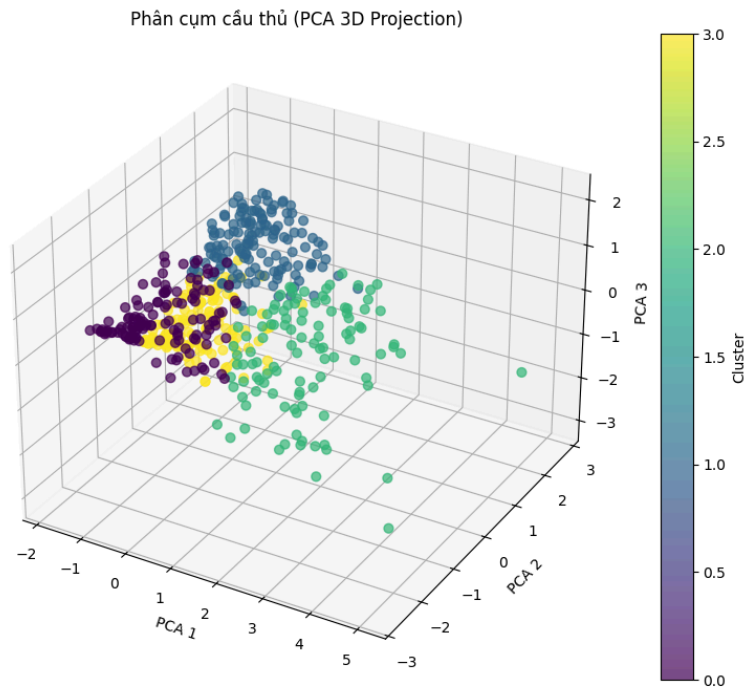
[VÍ DỤ] MỘT SỐ CẦU THỦ TIÊU BIỂU TRONG TỪNG NHÓM:

- Nhóm 0: Acheampong Joshua, Adingra Simon, Amass Harry, Antony, Araujo Julian...
- Nhóm 1: Adams Tyler, Aina Ola, Aït-Nouri Rayan, Akanji Manuel, Alexander-Arnold Trent...
- Nhóm 2: Alcaraz Carlos, Asensio Marco, Barkley Ross, Barnes Harvey, Bellegarde Jean-Ricner...
- Nhóm 3: Adarabioyo Tosin, Agbadou Emmanuel, Ajer Kristoffer, Aké Nathan, Almirón Miguel...

4.3. Giải thích Thuật toán Giảm chiều (PCA):

- Để có thể quan sát trực quan sự phân bố của 4 cụm trong không gian 4 chiều dữ liệu gốc là điều bất khả thi. Vì vậy, nhóm áp dụng thuật toán **PCA (Principal Component Analysis)**.
- PCA tính toán ma trận hiệp phương sai và tìm ra các giá trị riêng/vector riêng để tạo ra các "thành phần chính" (Principal Components) - là các trục tọa độ mới giữ lại được lượng thông tin (phương sai) lớn nhất từ dữ liệu gốc.
- Nhóm sử dụng `PCA(n_components=2)` và `PCA(n_components=3)` ép không gian 4 chiều xuống còn 2 và 3 chiều. Tọa độ mới được lưu vào các cột `PCA1`, `PCA2`, `PCA3`.
- Cuối cùng, dùng thư viện `matplotlib` kết hợp không gian `Axes3D` để vẽ scatter plot (biểu đồ phân tán). Các điểm dữ liệu được gán màu (hue/c) dựa trên nhãn `Cluster` vừa nhận được từ thuật toán K-Means, tạo ra hình ảnh trực quan thể hiện rõ mức độ tách biệt của các nhóm cầu thủ.





III. KẾT LUẬN

- **Khó khăn gặp phải:** Xử lý cơ chế chống bot của Fbref và FootballTransfers.
- **Kinh nghiệm rút ra:** Tối ưu hóa tốc độ cào dữ liệu bằng cách đọc thẳng JSON/Base64 thay vì mô phỏng thao tác người dùng. Áp dụng thành công các mô hình học máy cơ bản vào thực tế.
- **Hướng phát triển:** Xây dựng giao diện web thay vì CLI, thu thập thêm dữ liệu thẻ phạt/cản phá để mô hình định giá chính xác hơn cho hàng phòng ngự.